# How to brief the editor: the conductor’s PDF briefs, for another team

A guide for the team Dinis Cruz is assembling for the Web Summit, at his word (2 November 2026: “can you write a detailed brief for the other team … to know and understand how you’re doing these debriefs … so that we basically have … this capability there”). Written by conductor@book.the-ultimate-insider from the 35 briefs it has sent him between 30 October and 2 November 2026. Everything here is public, and so is everything it points at (github.com/SGit-AI/SGit-AI__Website__Book__The-Ultimate-Insider, branch `dev`).

## 1. What a brief is for

The editor runs a team of agents he cannot watch. A brief is how the team speaks to him, and the only thing he must read. It answers four questions, in this order:

1. **What needs me?** Decisions, each with its options and a recommendation, that he answers with a word.
2. **What happened?** In plain words, the punchline first, with the numbers that matter.
3. **What did you hear me say?** His own words quoted back, with what was done, so he can see he was understood.
4. **What next, and where did this come from?**

A brief is not a log. If he reads only the headline and the boxes, he should be able to decide.

## 2. When

- **After every run** of the conductor, scheduled or in his chat, while there is anything new. A quiet run gets a one-page brief that says so.
- **Numbered and dated**: brief 1, 2, 3 …, the number and date at the foot of every page. He asked for this so he can refer to “brief 16” and find it.
- **When he asks** for a picture of a period (the night, the day): a review brief with a chart drawn from the git log (section 6).

## 3. The rules that make it trusted

- **Nothing is decided for him.** A brief offers options and a recommendation; a decision is recorded only in his own words, quoted, with where and when he said them. Things relayed from another session are marked “relayed”.
- **His words verbatim.** When he speaks (often by voice), his words are kept exactly as given in a file of their own, then read. The reading is marked “not his words”.
- **Every number is traced.** Counts come from the record (the git log, the claims register, a member’s letter), never from memory. A reviewer checks each figure in the brief against its source before it goes out; when one was wrong, it was fixed and the fix noted.
- **The failures stay in.** A skipped step, a lost evening, a miscount: said plainly, with what was done about it.
- **Plain words.** Short sentences, British spelling, no em-dashes, no jargon he has not used himself. The headline is one paragraph he could read aloud.
- **Public by default.** No key, token, session id, contact detail or private fact about anyone, ever.

- **Reviewed before it goes.** Each brief goes out at the end of a run whose plan and diff were reviewed by two sub-agents (section 7).

## 4. The shape

One JSON file per brief; a small tool turns it into a self-contained HTML page and an A4 PDF. The fields, in the order they print:

| FIELD | WHAT IT HOLDS |
|---|---|
| `number`, `date` | printed at the foot of every page |
| `kicker`, `title`, `when`, `by` | the top: a small kicker, a title that is the news, when, and who |
| `headline` | one paragraph: the punchline, then what needs him |
| `stats` | up to four tiles: `{"n": "3 of 3", "label": "readers who would start tomorrow"}` |
| `for_you` | the decisions: `{"id", "ask", "detail", "options": [...], "recommend"}`, printed as cards with tick boxes; `{"group": "..."}` makes a heading |
| `you_asked` | his words and what was done: `{"at", "said", "done", "where", "status"}` |
| `team` | who is doing what: `{"id", "status", "doing"}` |
| `wide` | a full-width picture (a chart, a page from a draft): `{"title", "path", "caption"}` |
| `sections` | `{"title", "text"}` or `{"items": [...]}` or `{"table": {"head", "rows"}}` |
| `images` | smaller pictures in a grid, embedded, so the page stands alone |
| `next`, `sources` | what comes next; where every fact came from |

A worked example: `team/conductor/editor-briefs/2026-11-02T0220Z.json` (brief 35), and its PDF beside it.

## 5. The tool

`team/conductor/tools/editor_brief.py` (standard library only; about 160 lines):

```
python3 -I team/conductor/tools/editor_brief.py team/conductor/editor-briefs/<stamp>.json
```

It writes `<stamp>.html` and `<stamp>.pdf` beside the JSON, with the PDF from a headless Chromium (`--print-to-pdf`). What it does that matters:

- the CSS is inline, so the HTML is one file you can send; pictures are embedded as data URIs;
- a fixed footer prints “Conductor brief N · date · title” on every page;
- it refuses to write a page with an em-dash in it;
- it prints the page count, so a brief that has grown to four pages is noticed.

Copy it, change the kicker and the colours to your team’s, and keep the rest.

## 6. Pictures

The briefs he liked best had pictures that told the story on their own:

- **Who worked when:** a member-by-hour grid of commits from the git log (`team/conductor/tools/day_heatmap.py since until out.svg`): one hue, light to dark, the count in each cell, release commits on their own row. It made “the night” visible in one look.
- **The thing itself:** a page from a draft, a cover study, the options for a drawing, side by side, as a full-width picture (render the PDF page to PNG with `pdftoppm`).
- **Readers’ verdicts** as stat tiles (“6 of 6 read to the end”).

Every picture says where it came from in its caption.

## 7. The run that makes a brief

The brief is the last thing a run makes, not a separate job:

1. Pull, and read the new letters and commits since the last brief.
2. A planning sub-agent writes the plan; a security sub-agent reviews it (no decision taken for the editor, nothing private, nothing outside the conductor’s files).
3. Do the work: answer the letters, record any decision in his words, write the brief JSON, build it.
4. A security sub-agent reviews the diff and checks the brief’s numbers against their sources; a second sub-agent checks the work against the plan. Fix what they find; note the fix.
5. Push, publish the brief on the team’s site, and send him the PDF with one line saying what it is.
6. Reply in the chat in a few short paragraphs: what is new, and what needs him.

## 8. Lessons from 35 briefs

- **Lead with what needs him**, and keep it to three asks or fewer. Repeat an open ask in the next brief rather than letting it drop.
- **Recommend.** He decides faster when the conductor says which option it would pick and why, in one line.
- **Show, do not describe.** A cover study or a draft page beats a paragraph about it.
- **Quote him back.** “What you asked, and what was done” with his exact words caught misunderstandings early.
- **Say when nothing needs him.** “Nothing new beyond brief 16” is a useful brief.
- **Name the cause of a gap.** When the account’s usage limit stopped every wake for an evening, the brief said so, with the evidence, and asked how hard the team should run.
- **Correct in the open.** When a reviewer found a miscount, the brief was rebuilt and the commit said why.

## 9. Where to look

- The briefs: `team/conductor/editor-briefs/` (JSON, HTML and PDF of each; README of the fields).
- The tools: `team/conductor/tools/editor_brief.py`, `team/conductor/tools/day_heatmap.py`.

- How a run works: `team/conductor/WAKE.md` (the five steps) and the review briefs in `team/conductor/reviews/`.
- The site page that lists them: `/updates/` on the-ultimate-insider.book.sgit.ai.